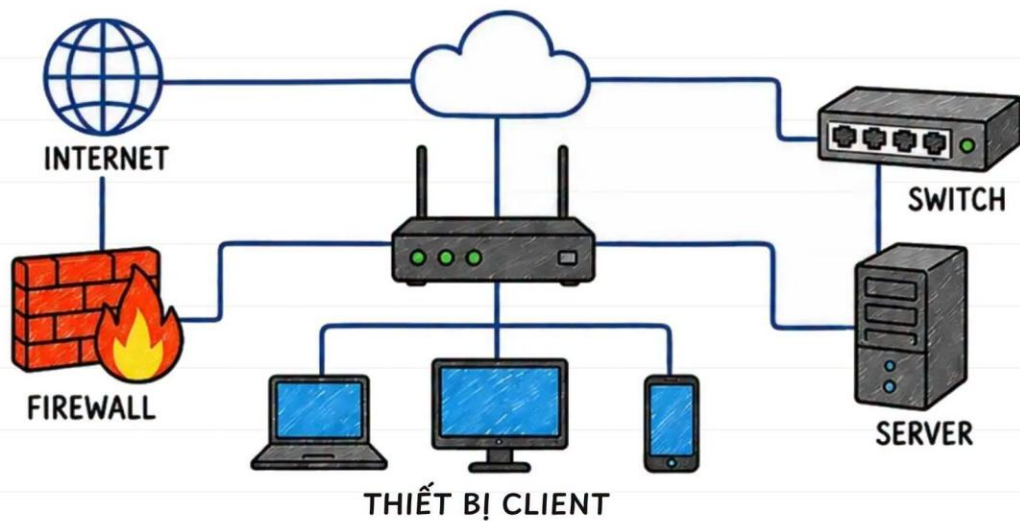


GHI CHÚ TOÀN TẬP

NETWORKING

≧ (Từ cơ bản đến nâng cao) ≦



≧ HỌC • HIỂU • ÁP DỤNG ≦



THEO DÕI



CHIA SẺ



BÌNH LUẬN



THÍCH

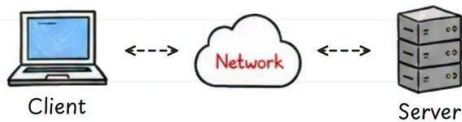
Giới thiệu về Computer Networking

1 Networking là gì?



Networking là quá trình kết nối hai hoặc nhiều thiết bị để trao đổi dữ liệu và chia sẻ tài nguyên thông qua các giao thức truyền thông (communication protocols).

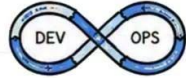
3 Kiến trúc Client ↔ Server



- ✦ Client (người dùng/thiết bị) gửi request.
- ✦ Server xử lý request.
- ✦ Server gửi lại response.

2 Vì sao DevOps cần Networking?

- ✓ Cho phép giao tiếp giữa các tool, service và người dùng
- ✓ Cực kỳ quan trọng với CI/CD, monitoring, logging & automation
- ✓ Đảm bảo bảo mật, hiệu năng, khả năng mở rộng và độ tin cậy
- ✓ Là nền tảng của Cloud, Container & Microservices



4 LAN, WAN & Internet



LAN (Local Area Network)

Kết nối các thiết bị trong phạm vi nhỏ như nhà ở, văn phòng, khuôn viên trường.



WAN (Wide Area Network)

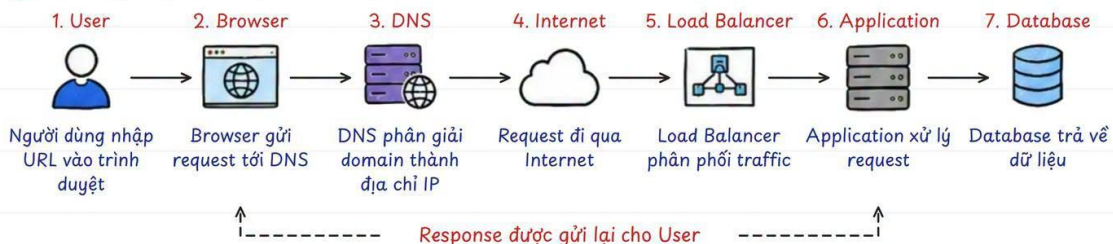
Kết nối các mạng LAN qua khoảng cách lớn như giữa các thành phố, quốc gia.



Internet

Mạng toàn cầu gồm nhiều mạng, kết nối cả thế giới.

5 Luồng request thực tế (Ví dụ)



6 Networking được dùng ở đâu trong DevOps?



Hạ tầng Cloud



Containers & Docker



Kubernetes Services



CI/CD Pipelines



Monitoring & Logging



Bảo mật & Firewall Rules



Giao tiếp giữa các Microservices

★ Mẹo phỏng vấn

Networking là một trong những chủ đề được hỏi nhiều nhất trong các buổi phỏng vấn DevOps.

💡 Ghi nhớ

Mọi request của ứng dụng đều phải đi qua networking trước khi đến được server.

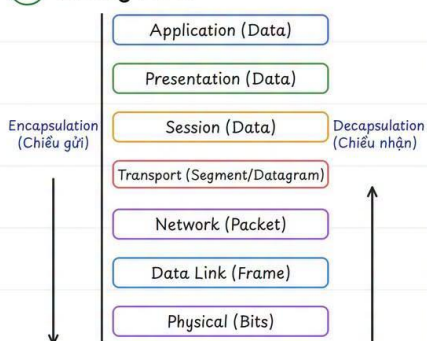
Mô hình OSI (7 tầng)

Mô hình OSI (Open Systems Interconnection) là một khung khái niệm (conceptual framework) giúp chuẩn hóa cách dữ liệu được truyền qua mạng.

1 Tổng quan 7 tầng

Tầng (No.)	Tên tầng	Nhiệm vụ	Đơn vị dữ liệu (PDU)	Thiết bị ở tầng này	Ví dụ / Protocol
7 	Application Layer	Cung cấp dịch vụ mạng cho ứng dụng của người dùng.	Data	-	HTTP, HTTPS, FTP, SMTP, DNS, SSH, DHCP
6 	Presentation Layer	Xử lý định dạng dữ liệu, mã hóa (encryption), nén (compression), chuyển đổi (translation).	Data	-	SSL/TLS, JPEG, ASCII, MPEG
5 	Session Layer	Quản lý session: thiết lập, duy trì & kết thúc kết nối.	Data	-	NetBIOS, RPC, PPTP
4 	Transport Layer	Truyền dữ liệu end-to-end, phân đoạn (segmentation), flow control, error control.	Segment (TCP) Datagram (UDP)	L4 Switch (Load Balancer)	TCP, UDP
3 	Network Layer	Đánh địa chỉ logic, routing, chọn đường đi, chuyển tiếp packet.	Packet	Router (L3)	IP (IPv4, IPv6), ICMP, OSPF, BGP
2 	Data Link Layer	Framing, đánh địa chỉ MAC, phát hiện lỗi, kiểm soát truy cập.	Frame	Switch / Bridge (L2)	Ethernet, PPP, 802.11, MAC
1 	Physical Layer	Truyền các bit thô qua môi trường vật lý.	Bits	Hub, Repeater	Cáp, đầu nối, Hub, chuẩn tín hiệu

2 Luồng PDU



3 Mẹo ghi nhớ

Please Do Not Throw Sausage Pizza Away

P - Physical
D - Data Link
N - Network
T - Transport
S - Session
P - Presentation
A - Application
(đọc từ tầng 1 lên tầng 7)

4 Điểm cần nhớ

- ✓ 3 tầng trên (5-7) tập trung vào giao tiếp của ứng dụng.
- ✓ Tầng giữa (4) đảm bảo truyền end-to-end tin cậy.
- ✓ 2 tầng dưới (1-2) đảm nhận việc truyền dữ liệu thực tế.
- ✓ Dữ liệu đi xuống từ tầng 7 đến 1 khi gửi và đi lên từ tầng 1 đến 7 khi nhận.



Ghi nhớ

Mô hình OSI là mô hình tham chiếu (reference model). Trên thực tế, các mạng sử dụng mô hình TCP/IP.



Mẹo phỏng vấn

Hãy sẵn sàng giải thích từng tầng thật dễ hiểu kèm ví dụ và thiết bị. Nhớ kỹ mẹo ghi nhớ để không bị rối khi căng thẳng!

Địa chỉ IP (IP Addressing)

1 IPv4 vs IPv6

IPv4	IPv6
- Địa chỉ 32-bit - Định dạng: a.b.c.d (mỗi octet từ 0 - 255) - Ví dụ: 192.168.1.10 - Tổng số địa chỉ: ~4,3 tỷ	- Địa chỉ 128-bit - Định dạng: xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx - Ví dụ: 2001:0db8:85a3:0000:0000:8a2e:0370:7334 - Tổng số địa chỉ: ~340 undecillion ($\approx 3,4 \times 10^{38}$)

💡 IPv6 ra đời để khắc phục tình trạng cạn kiệt địa chỉ IPv4, đồng thời mang lại khả năng mở rộng & bảo mật tốt hơn.

2 Public vs Private IP



Public IP (truy cập được từ Internet)

- Do ISP cấp
- Duy nhất trên toàn cầu
- Dùng để giao tiếp qua Internet
- Ví dụ: 8.8.8.8



Private IP (dùng nội bộ)

- Dùng trong mạng riêng (LAN/VPC)
- Không thể route trên Internet
- Giúp tiết kiệm Public IP
- Ví dụ: 10.0.0.0/8 172.16.0.0/12 192.168.0.0/16

3 Static vs Dynamic IP

Static IP

- Cấu hình thủ công
- Không thay đổi
- Dùng cho server, thiết bị cần truy cập ổn định
- Ví dụ: 49.36.12.5



Dynamic IP

- Được DHCP cấp tự động
- Có thể thay đổi theo thời gian
- Dùng cho các thiết bị client
- Ví dụ: Laptop của developer (thay đổi định kỳ)

4 Loopback Address

- Được thiết bị dùng để giao tiếp với chính nó.
- Dùng để test và phát triển ở local.
- IPv4 Loopback: 127.0.0.1
- IPv6 Loopback: ::1



✓ Packet gửi tới loopback không bao giờ rời khỏi thiết bị.

5 CIDR (Classless Inter-Domain Routing) - Giới thiệu

- CIDR là phương pháp cấp phát địa chỉ IP và routing các IP packet.
- Thay thế cho cách đánh địa chỉ theo lớp (classful addressing) truyền thống.
- Ký hiệu: IP Address / Prefix Length
- Ví dụ: 192.168.1.0/24
 - ✓ 24 nghĩa là 24 bit đầu là phần network
 - ✓ 8 bit còn lại dành cho host
- CIDR giúp sử dụng IP hiệu quả và gộp route (route summarization).

★ Mẹo phỏng vấn

Hãy hiểu thật kỹ các loại IP, dải IP và CIDR - đây là những câu hỏi rất hay gặp trong phỏng vấn DevOps & Networking!

192.168.1.0/24	
Network (24 bits)	Host (8 bits)
192 . 168 . 1	0
Hỗ trợ $2^8 - 2 = 254$ host khả dụng	
Các dải CIDR phổ biến	/8 → 16,777,214 host
	/16 → 65,534 host
	/24 → 254 host
	/30 → 2 host (Point-to-Point)

💡 Ghi nhớ

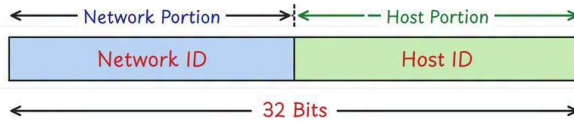
Địa chỉ IP là nền tảng của mọi hoạt động networking. Không có IP, các thiết bị không thể giao tiếp với nhau!

Subnetting & CIDR

1 Thuật ngữ chính

- **Network ID** : Xác định phần network của một địa chỉ IP.
- **Host ID** : Xác định thiết bị cụ thể trong network đó.
- **Subnet Mask** : Chia IP thành phần network và phần host.
- **CIDR Notation** : Cách viết ngắn gọn, hiệu quả để biểu diễn IP và subnet mask.
- **Available Hosts** : Tổng số địa chỉ IP khả dụng trong một subnet.

2 Cấu trúc địa chỉ IP (IPv4)



Ví dụ:

IP Address : 192.168.1.10 Subnet Mask : 255.255.255.0 (/24)

IP Address : 11000000.10101000.00000001.00001010

Subnet Mask : 11111111.11111111.11111111.00000000

Network ID (24 bits)
Host ID (8 bits)

3 Subnet Mask

Subnet mask là một số 32-bit cho biết phần nào của IP là Network ID và phần nào là Host ID.

Các Subnet Mask phổ biến

CIDR	Subnet Mask	Network / Host Bits
/8	255.0.0.0	8 / 24
/16	255.255.0.0	16 / 16
/24	255.255.255.0	24 / 8
/25	255.255.255.128	25 / 7
/26	255.255.255.192	26 / 6
/27	255.255.255.224	27 / 5
/28	255.255.255.240	28 / 4
/30	255.255.255.252	30 / 2

4 CIDR Notation

CIDR (Classless Inter-Domain Routing) loại bỏ việc phải dùng mạng theo lớp (classful) và cho phép chia subnet với độ dài thay đổi (variable length subnetting).

Định dạng: IP Address / Prefix Length

Ví dụ:

- 192.168.1.0/24 → 24 bit đầu là phần network
- 10.0.0.0/8 → 8 bit đầu là phần network
- 172.16.0.0/16 → 16 bit đầu là phần network

★ Mẹo phỏng vấn

CIDR giúp cấp phát IP hiệu quả, gộp route (route summarization) và giảm kích thước routing table trên Router.

5 Tính số host khả dụng

Công thức:

Tổng số host = 2^n

Host khả dụng = $2^n - 2$

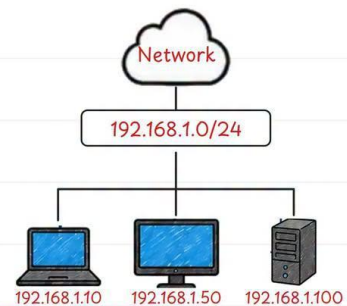
Trong đó 'n' = số bit host (trừ 2 cho Network ID và Broadcast ID)

Ví dụ tính toán

Cho IP : 192.168.1.10/24

Subnet Mask : 255.255.255.0

CIDR	/24
Host Bits (n)	8
Tổng số host (2^n)	$2^8 = 256$
Host khả dụng ($2^n - 2$)	$256 - 2 = 254$
Host Range	192.168.1.1 - 192.168.1.254
Network ID	192.168.1.0
Broadcast ID	192.168.1.255



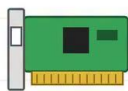
💡 Ghi nhớ

- ✓ Network ID → IP đầu tiên trong subnet (tất cả host bit = 0)
- ✓ Broadcast ID → IP cuối cùng trong subnet (tất cả host bit = 1)

Địa chỉ MAC & ARP

1 MAC Address (Media Access Control)

- Địa chỉ phần cứng duy nhất được gán cho card mạng (Network Interface Card - NIC).
- Hoạt động ở Data Link Layer (Layer 2).
- 48-bit address (6 nhóm, mỗi nhóm 2 chữ số hex).
- Định dạng: 00:1A:2B:3C:4D:5E
- Định danh duy nhất một thiết bị trong mạng local.



Ví dụ MAC
00:1A:2B:3C:4D:5E

2 Physical vs Logical Address

Physical Address (MAC)

- Địa chỉ phần cứng
- Layer 2 (Data Link)
- Do nhà sản xuất gán
- Dùng cho giao tiếp trong mạng local

Ví dụ:

00:1A:2B:3C:4D:5E

Logical Address (IP)

- Địa chỉ phần mềm
- Layer 3 (Network)
- Gán thủ công hoặc qua DHCP
- Dùng cho giao tiếp end-to-end

Ví dụ:

192.168.1.10

3 ARP (Address Resolution Protocol)

- ARP ánh xạ địa chỉ IP (Logical) sang địa chỉ MAC (Physical).
- Hoạt động trong mạng local.
- Nếu máy gửi chưa biết MAC của IP đích, nó sẽ gửi ARP Request (Broadcast).
- Máy đích trả lời bằng ARP Reply (Unicast) chứa địa chỉ MAC của nó.



ARP giúp tìm địa chỉ MAC của một thiết bị dựa vào IP của nó.

Ghi nhớ

ARP giống như hỏi cả mạng: "Ai đang dùng 192.168.1.10? Hãy báo lại cho 192.168.1.20."

4 Quy trình ARP (Broadcast & Reply)

1. ARP Request (Broadcast)

"Ai đang dùng 192.168.1.10?
Hãy báo lại cho 192.168.1.20"



Máy gửi
192.168.1.20



Máy đích
192.168.1.10

2. ARP Reply (Unicast)

"192.168.1.10 có địa chỉ MAC là
00:1A:2B:3C:4D:5E"

5 MAC Table của Switch

- Switch học địa chỉ MAC từ các frame gửi đến.
- Lưu vào MAC Address Table kèm thông tin port.
- Dùng để chuyển frame tới đúng port (không broadcast).

Ví dụ MAC Table

MAC Address	Port
00:1A:2B:3C:4D:5E	Port 1
00:1A:2B:3C:4D:6F	Port 2
00:1A:2B:3C:4D:7A	Port 3
00:1A:2B:3C:4D:8B	Port 4
...	...

Điểm chính

- ✓ MAC = Địa chỉ phần cứng (Layer 2)
- ✓ IP = Địa chỉ logic (Layer 3)
- ✓ ARP ánh xạ IP → MAC
- ✓ Switch dùng MAC Table để chuyển tiếp thông minh

6 Luồng chuyển packet



IP: 192.168.1.20
MAC: 00:1A:2B:3C:4D:5E



IP: 192.168.1.10
MAC: 00:1A:2B:3C:4D:6F

- Máy gửi kiểm tra IP đích có cùng subnet hay không.
- Dùng ARP để lấy MAC của máy đích.
- Gửi frame tới Switch (kèm MAC đích).
- Switch tra MAC Table và chuyển tiếp tới đúng port.
- Máy đích nhận frame và xử lý.

Mẹo phòng vấn

Hãy nắm rõ sự khác nhau giữa MAC & IP, cách ARP hoạt động và cách switch học & chuyển tiếp frame. Đây là những câu hỏi thường gặp trong phỏng vấn DevOps, Network & Cloud.

TCP vs UDP

TCP và UDP là các protocol ở Transport Layer, dùng để truyền dữ liệu giữa các ứng dụng qua mạng IP.

1 Connection-Oriented vs Connectionless

• TCP là Connection-Oriented

Kết nối được thiết lập trước khi truyền dữ liệu và được đóng lại sau khi hoàn tất.

• UDP là Connectionless

Không cần thiết lập kết nối. Dữ liệu được gửi đi trực tiếp mà không cần tạo session.

3 Reliability vs Speed

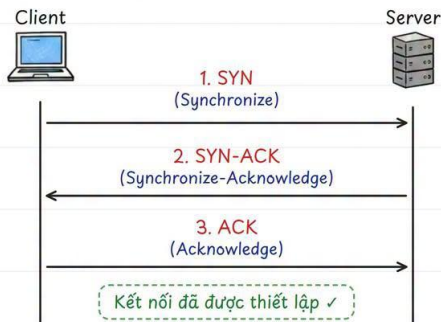
TCP

- ✓ Truyền dữ liệu tin cậy
- ✓ Đảm bảo dữ liệu đến đúng thứ tự
- ✓ Kiểm tra lỗi & truyền lại (retransmission)
- ✓ Chậm hơn do overhead

UDP

- ✓ Best-effort delivery (không đảm bảo)
- ✓ Không đảm bảo thứ tự hay việc nhận được
- ✓ Không truyền lại, không khôi phục lỗi
- ✓ Nhanh và nhẹ

2 Three-Way Handshake (TCP)



4 Ví dụ thực tế

TCP (ứng dụng cần tin cậy)

- Duyệt web (HTTP/HTTPS)
- Email (SMTP, IMAP, POP3)
- Truyền file (FTP, SFTP)
- Kết nối database (MySQL, PostgreSQL)

UDP (ứng dụng cần tốc độ)

- Game online
- Video streaming (live)
- Voice over IP (VoIP)
- DNS Queries

5 So sánh TCP vs UDP

Tiêu chí	TCP	UDP
Loại	Connection-Oriented	Connectionless
Thiết lập kết nối	Bắt buộc (Three-way Handshake)	Không cần
Độ tin cậy	Tin cậy (ACK + Retransmission)	Best-effort (không đảm bảo)
Thứ tự	Giữ đúng thứ tự	Không đảm bảo thứ tự
Kiểm tra lỗi	Có (Checksum, ACK)	Có (chỉ Checksum)
Truyền dữ liệu	Đảm bảo	Không đảm bảo
Tốc độ	Chậm hơn	Nhanh hơn
Overhead	Cao hơn (Header + Handshake)	Thấp hơn (Header tối giản)
Kiểu dữ liệu	Stream (liên tục)	Datagram (packet độc lập)
Header Size	20-60 Bytes	8 Bytes
Use Cases	Web, Email, File Transfer, DB, SSH	Gaming, Streaming, DNS, VoIP, IoT

Điểm cần nhớ

- TCP giống như một cuộc gọi điện thoại (có kết nối, tin cậy, đúng thứ tự).
- UDP giống như bộ đàm walkie-talkie (không cần thiết lập, nhanh, có thể mất packet).



★ Mẹo phỏng vấn

Nếu chấp nhận mất dữ liệu nhưng tốc độ là quan trọng → dùng **UDP**.

Nếu độ chính xác và độ tin cậy là quan trọng → dùng **TCP**.

💡 Ghi nhớ

TCP = Trust, Confirm, Protect (Tin cậy, Xác nhận, Bảo vệ)

UDP = User, Datagram, Performance (User Datagram Protocol, ưu tiên hiệu năng)

Port & các giao thức phổ biến

Port là các điểm cuối logic (logical endpoint) được protocol sử dụng để xác định ứng dụng và service trên một host.

Protocol / Service	Port	Transport	Mục đích
 HTTP (HyperText Transfer Protocol)	80	TCP	Truyền dữ liệu web không mã hóa
 HTTPS (HTTP Secure)	443	TCP	Truyền dữ liệu web được mã hóa (SSL/TLS)
 SSH (Secure Shell)	22	TCP	Đăng nhập từ xa & thực thi lệnh an toàn
 DNS (Domain Name System)	53	UDP/TCP	Phân giải tên miền (domain name)
 FTP (File Transfer Protocol)	21 (Control) 20 (Data)	TCP	Truyền file (không mã hóa)
 SFTP (SSH File Transfer Protocol)	22	TCP	Truyền file an toàn qua SSH
 SMTP (Simple Mail Transfer Protocol)	25	TCP	Gửi email
Database Ports			
 MySQL	3306	TCP	MySQL Database
 PostgreSQL	5432	TCP	PostgreSQL Database
 MongoDB	27017	TCP	MongoDB Database
 Redis	6379	TCP	Redis Cache / Key-Value Store

✓ Điểm chính

- ✓ Port + địa chỉ IP xác định duy nhất một service.
- ✓ Well-known ports : 0 - 1023
- ✓ Registered ports : 1024 - 49151
- ✓ Dynamic / Private ports : 49152 - 65535

★ Mẹo phòng vấn

Luôn nhắc đến protocol đi kèm với port (ví dụ: 80/TCP, 53/UDP).
Firewall rule được xây dựng dựa trên IP, port và protocol.

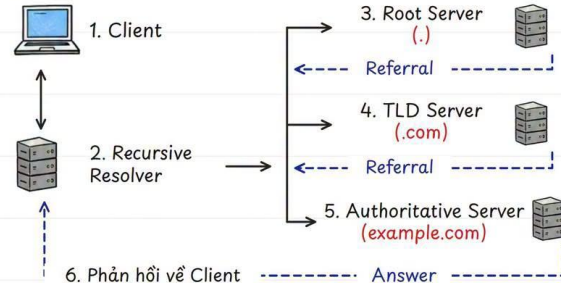
DNS (Domain Name System)

1 DNS là gì?

DNS là "danh bạ điện thoại" của Internet. Nó chuyển tên miền dễ đọc với con người thành địa chỉ IP mà máy tính dùng để giao tiếp.



2 Luồng phân giải DNS



💡 Caching giúp tăng tốc các lần tra cứu sau.

3 Các loại DNS Record quan trọng

Record Type	Mục đích	Ví dụ	Giải thích
A Record (Address)	Ảnh xạ tên miền tới địa chỉ IPv4.	example.com → 93.184.216.34	Trở tên miền tới một địa chỉ IPv4.
CNAME Record (Canonical Name)	Ảnh xạ một tên miền tới một tên miền khác.	www.example.com → example.com	Bí danh (alias) của một tên miền khác. Không trở tới IP.
MX Record (Mail Exchange)	Chỉ định mail server cho tên miền.	example.com → mail.example.com (Priority 10)	Cho biết nơi nhận email của tên miền.
TXT Record (Text)	Lưu thông tin dạng text về tên miền.	example.com → "v=spf1 include:_spf.example.com ~all"	Dùng cho SPF, xác minh quyền sở hữu, policy, v.v.

4 Ví dụ trong DevOps

Ứng dụng được host trên AWS.



💡 Vì sao DNS quan trọng trong DevOps?

- ✓ Ảnh xạ domain tới load balancer / service
- ✓ Hỗ trợ triển khai blue-green / canary
- ✓ Kết hợp với CDN, monitoring, email, v.v.

5 Tra cứu nhanh (Tóm tắt DNS Record)

Type	Use Case
A	Domain → IPv4 address
AAAA	Domain → IPv6 address
CNAME	Bí danh domain (trở tên này sang tên khác)
MX	Cấu hình mail server
TXT	SPF, DKIM, xác minh, policy
NS	Authoritative name server
PTR	Reverse DNS lookup (IP → Domain)

★ Mẹo phòng vấn

Luôn hiểu rõ cách DNS hoạt động, các loại record và cách caching ảnh hưởng đến thời gian phân giải.

HTTP & HTTPS

1 Vòng đời của một request (Request Lifecycle)



💡 Keep-Alive cho phép gửi nhiều request trên cùng một TCP connection (hiệu năng tốt hơn).

2 HTTP Methods

Method	Mục đích	Idempotent	Safe
GET	Lấy dữ liệu	✓	✓
POST	Tạo resource mới	✗	✗
PUT	Cập nhật toàn bộ resource	✓	✗
PATCH	Cập nhật một phần resource	✗	✗
DELETE	Xóa resource	✓	✗
HEAD	Chỉ lấy header	✓	✓
OPTIONS	Lấy các method được hỗ trợ	✓	✓

3 HTTP Status Codes (quan trọng nhất)

1xx	Informational	Đã nhận request, đang tiếp tục xử lý
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

4 SSL/TLS (Cách HTTPS hoạt động)



1. Client Hello (hỗ trợ TLS versions, ciphers) →
2. Server Hello + Certificate (server gửi certificate) ←
3. Key Exchange (client xác minh certificate, gửi key đã được mã hóa) →
4. Secure Session Established (truyền dữ liệu được mã hóa) ↔

💡 **Điểm chính**

- ✓ Dùng mã hóa để bảo vệ dữ liệu khi truyền (in transit).
- ✓ Đảm bảo xác thực bằng SSL Certificate.
- ✓ Ports: HTTP = 80
HTTPS = 443

5 REST APIs

- REST (Representational State Transfer) là một kiến trúc (architectural style) dành cho API.
- Dùng các HTTP method để thao tác trên resource.
- Resource được xác định bằng URL.
- Định dạng dữ liệu phổ biến: JSON / XML.

Ví dụ:

GET	/api/users	→ Lấy tất cả users
POST	/api/users	→ Tạo user
GET	/api/users/123	→ Lấy user theo ID
PUT	/api/users/123	→ Cập nhật user
DELETE	/api/users/123	→ Xóa user

6 Lợi ích của HTTPS

- ✓ Mã hóa dữ liệu → chống nghe lén (eavesdropping).
- ✓ Xác thực server → chống tấn công man-in-the-middle.
- ✓ Đảm bảo toàn vẹn dữ liệu → dữ liệu không thể bị sửa đổi.
- ✓ Tăng niềm tin của người dùng (biểu tượng ổ khóa trên trình duyệt).
- ✓ Bắt buộc với web app, API, thanh toán, đăng nhập hiện đại.

🔒 https://example.com



7 Tra cứu nhanh

Protocol	HTTP (80) / HTTPS (443)
Mục đích	Giao tiếp trên web
Dữ liệu	HTTP → Plain Text HTTPS → Encrypted (TLS)
Bảo mật	HTTP → Không bảo mật HTTPS → Bảo mật
Best Practice	Luôn dùng HTTPS trên production



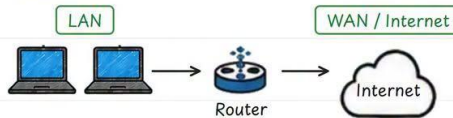
Mẹo phỏng vấn:

Hãy giải thích được vòng đời request, sự khác nhau giữa HTTP & HTTPS và khi nào dùng từng HTTP method.

Routing, NAT & Gateway

1 Router (chức năng)

- Router kết nối nhiều mạng và chuyển tiếp packet dựa trên địa chỉ IP.
- Router dùng routing table để quyết định đường đi tốt nhất.



2 Routing (cách packet tìm đường)

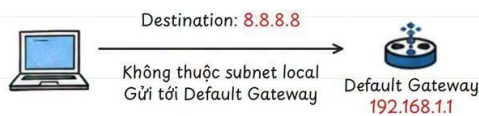
Routing Table (Ví dụ)

Destination Network	Next Hop
192.168.1.0/24	Directly Connected
10.0.0.0/24	10.0.0.1
0.0.0.0/0	203.0.113.1

Điểm chính
0.0.0.0/0 là Default Route. Dùng cho mọi đích đến không xác định (Internet).

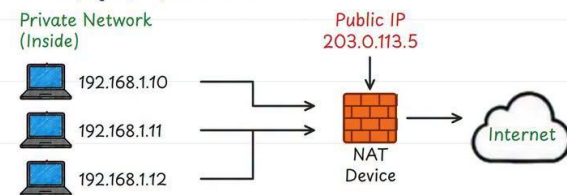
3 Default Gateway

- Default gateway là địa chỉ IP của router trong mạng local của bạn.
- Nếu IP đích nằm ngoài subnet local, traffic sẽ được gửi tới default gateway.



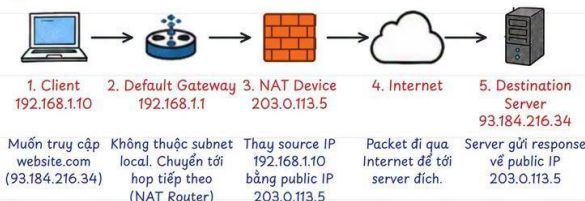
4 NAT (Network Address Translation)

- NAT cho phép nhiều Private IP dùng chung một Public IP.
- Ẩn mạng nội bộ khỏi Internet.



Các loại:
SNAT (Source NAT) → cho traffic đi ra (outgoing)
DNAT (Destination NAT / Port Forwarding) → cho traffic đi vào (incoming)

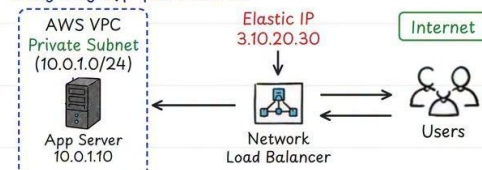
5 Hành trình packet (Inside → Internet)



Đường về:
Server → Internet → NAT (203.0.113.5) → 192.168.1.10 (Client ban đầu)

6 Ví dụ thực tế

Ứng dụng được host trong AWS VPC và được người dùng truy cập qua Internet.



- Users truy cập `https://example.com`
- DNS phân giải tới 3.10.20.30 (Elastic IP).
- Load Balancer route traffic tới App Server (10.0.1.10) trong private subnet.
- Khi App Server cần ra Internet (cập nhật, tải package), traffic đi qua NAT Gateway.

★ Mẹo phỏng vấn

- Router quyết định đường đi (Routing).
- Default Gateway = "cửa ra" tới các mạng khác.
- NAT ẩn Private IP và cho phép truy cập Internet.
- Hiểu luồng packet end-to-end.

💡 Ghi nhớ

- Private IP: 10.x.x.x, 172.16-31.x.x, 192.168.x.x
- Default Gateway = IP của Router trong subnet
- NAT ≠ Routing (NAT chuyển đổi địa chỉ, Router định tuyến)

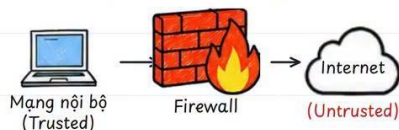
🔧 Lệnh thường dùng

Xem Route Table : `ip route (Linux)`
Kiểm tra Default Gateway : `ip route | grep default`
Kiểm tra kết nối : `ping 8.8.8.8`
Trace route : `tracert example.com`

Firewall & bảo mật mạng

1 Firewall là gì?

Firewall giám sát và kiểm soát network traffic đi vào và đi ra dựa trên các security rule.



Firewall đóng vai trò rào chắn giữa mạng nội bộ đáng tin cậy (trusted) và mạng bên ngoài không đáng tin cậy (untrusted).

2 Stateful vs Stateless Firewall

	Stateless Firewall	Stateful Firewall
	Kiểm tra từng packet một cách độc lập.	Theo dõi trạng thái của các kết nối đang hoạt động.
	Hoạt động ở L3/L4 (IP, Port, Protocol).	Hoạt động ở L3/L4 và lưu bảng trạng thái kết nối (connection state table).
	Nhanh hơn nhưng kém an toàn hơn.	Chậm hơn một chút nhưng an toàn hơn.
	Cho phép/chặn chỉ dựa trên rule.	Tự động cho phép traffic phản hồi (return traffic) của các kết nối hợp lệ.
	Ví dụ: Basic ACL, NACL.	Ví dụ: hầu hết firewall hiện đại, AWS Security Group.

3 Security Groups (Stateful)

- Firewall ảo dành cho các instance.
- Stateful: Return traffic được tự động cho phép.
- Rule chỉ có allow.
- Hoạt động ở cấp instance.
- Ví dụ: AWS Security Group, Azure NSG.

Ví dụ (Allow Rules)

Type	Protocol	Port	Source
SSH	TCP	22	203.0.113.10/32
HTTP	TCP	80	0.0.0.0/0
HTTPS	TCP	443	0.0.0.0/0

Chỉ traffic được khai báo mới được phép. Mọi traffic khác đều bị chặn.

4 Network ACL (NACL) (Stateless)

- Hoạt động ở cấp subnet (không phải instance).
- Stateless: phải cho phép cả inbound lẫn outbound.
- Rule có cả allow và deny.
- Được đánh giá theo thứ tự (rule number nhỏ nhất trước).
- Ví dụ: AWS NACL, ACL trên router truyền thống.

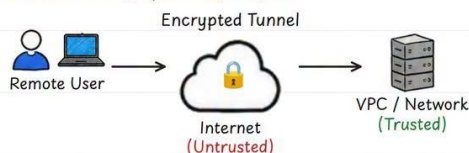
Ví dụ (Inbound Rules)

Rule #	Type	Protocol	Port	Source	Action
100	HTTP	TCP	80	0.0.0.0/0	ALLOW
110	SSH	TCP	22	203.0.113.10/32	ALLOW
120	ALL	ALL	ALL	0.0.0.0/0	DENY

Nếu packet không khớp rule ALLOW nào, nó sẽ bị DENY.

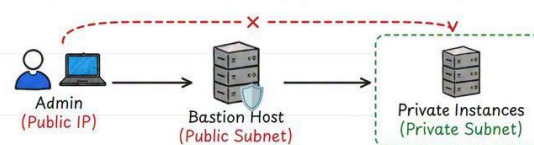
5 VPN (Virtual Private Network)

- Tạo một tunnel được mã hóa đi qua Internet.
- Dùng để truy cập từ xa an toàn hoặc kết nối Site-to-Site.
- Các loại: Site-to-Site VPN, Client-to-Site VPN.
- Protocol: IPsec, OpenVPN, SSL/TLS.



6 Bastion Host (Jump Server)

- Một gateway host an toàn để truy cập các private server.
- Truy cập trực tiếp tới private instance bị chặn.
- Admin SSH/RDP vào Bastion trước, sau đó mới vào server nội bộ.
- Best practice: bật MFA, logging, quyền tối thiểu (least privilege).



★ Điểm chính

- ✓ Dùng least privilege - chỉ cho phép traffic cần thiết.
- ✓ Ưu tiên stateful firewall (SG) cho đơn giản.
- ✓ Dùng NACL để kiểm soát ở cấp subnet & thêm một lớp bảo vệ.
- ✓ Luôn mã hóa traffic bằng VPN/TLS.
- ✓ Dùng Bastion Host - không bao giờ expose private server.

💡 Mẹo phòng vấn

- ✓ Stateful firewall ghi nhớ các kết nối, stateless thì không.
- ✓ Security Group = stateful, NACL = stateless.
- ✓ Mặc định: SG chặn mọi inbound, cho phép mọi outbound. Default NACL cho phép tất cả cả hai chiều (bạn phải tự deny những gì không muốn).
- ✓ VPN và Bastion Host giúp tăng cường bảo mật trên cloud.

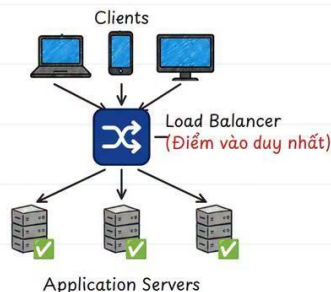
💻 Lệnh thường dùng

```
Xem rule iptables : sudo iptables -L -n -v
Trạng thái UFW : sudo ufw status
Mở port 443 : sudo ufw allow 443/tcp
Xem Security Group : aws ec2 describe-security-groups
```

Load Balancer & Proxy

1 Vì sao cần Load Balancer?

- Phân phối traffic đến tới nhiều server.
- Tăng tính sẵn sàng (availability), khả năng mở rộng và hiệu năng.
- Ngăn chặn điểm lỗi đơn (single point of failure).
- Thực hiện health check và chỉ route tới các server khỏe mạnh.



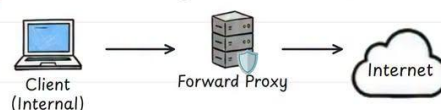
Điểm chính

Load balancer ẩn các backend server, mang lại tính sẵn sàng cao (high availability) và khả năng mở rộng liền mạch.

2 L4 vs L7 Load Balancing

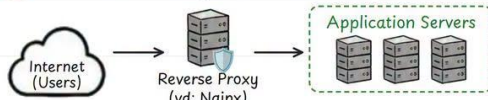
Tiêu chí	L4 (Transport Layer)	L7 (Application Layer)
Tầng	OSI Layer 4 (TCP/UDP)	OSI Layer 7 (HTTP/HTTPS)
Cơ sở	IP, Port, Protocol	Nội dung (URL, Host, Header)
Tốc độ	Rất nhanh	Chậm hơn một chút
Ví dụ sử dụng	Database, ứng dụng TCP, Microservices	Web app, API, SSL offloading, Content routing
Công cụ	AWS NLB, HAProxy (TCP)	AWS ALB, Nginx, Envoy

3 Forward Proxy (Client-side Proxy)



- ✦ Thay mặt client truy cập Internet.
- ✦ Ẩn IP nội bộ khỏi các website bên ngoài.
- ✦ Dùng cho caching, lọc nội dung, kiểm soát truy cập.

4 Reverse Proxy (Server-side Proxy)



- Đứng trước các backend server.
- Nhận request từ client, chuyển tiếp tới server phù hợp.
- Cung cấp SSL termination, caching, nén (compression), bảo mật.
- Ẩn backend server khỏi client.

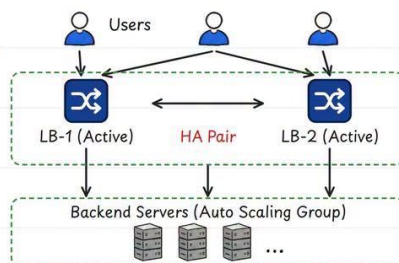
5 Nginx (Reverse Proxy / Load Balancer phổ biến)



- ✓ Hiệu năng cao, nhẹ và open-source.
- ✓ Hoạt động như Web Server, Reverse Proxy, Load Balancer, API Gateway.
- ✓ Hỗ trợ load balancing L4 & L7.
- ✓ SSL termination, caching, compression.
- ✓ Được dùng rộng rãi trong DevOps, Microservices, Kubernetes Ingress Controller.

6 High Availability (HA)

- Nhiều load balancer (Active-Active)
- Health check loại bỏ các node không khỏe
- Auto scaling tự động thêm/bớt server
- Dùng DNS hoặc Anycast IP để dự phòng LB
- Backup, monitoring và alert là bắt buộc



Ghi nhớ

- ✓ HA = Không có Single Point of Failure
- ✓ Dùng nhiều thành phần ở mọi tầng (LB, App, DB, Network).
- ✓ Thiết kế để chịu được lỗi (design for failure), không chỉ vì hiệu năng.

Mẹo phỏng vấn

Nắm rõ sự khác nhau giữa L4 & L7.
Hiểu khi nào dùng Proxy, khi nào dùng Load Balancer.
Chuẩn bị sẵn ví dụ: Nginx, HAProxy, AWS ALB/NLB.

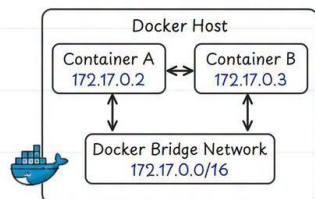
Công cụ phổ biến

Load Balancer: Nginx, HAProxy, AWS ALB/NLB, GCP Cloud Load Balancer, Azure Load Balancer
Reverse Proxy: Nginx, Traefik, Envoy
Forward Proxy: Squid, Nginx

Docker Networking

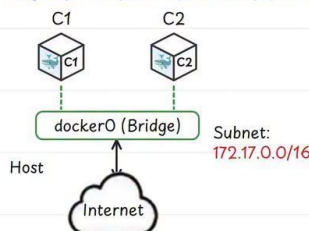
1 Tổng quan Docker Network

- Docker cung cấp nhiều network driver giúp container giao tiếp an toàn và hiệu quả.
- Dùng docker network ls để liệt kê các network.



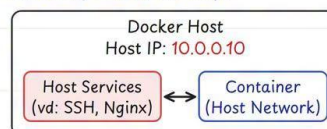
2 Bridge Network (mặc định)

- Chế độ network mặc định.
- Container kết nối qua một bridge ảo (docker0) trên host.
- Container có thể giao tiếp với nhau và truy cập mạng bên ngoài thông qua NAT.



3 Host Network

- Container dùng chung network namespace của host.
- Không cách ly - container dùng luôn IP của host.
- Hiệu năng cao, độ trễ thấp.



Dùng khi:

Ứng dụng cần hiệu năng cao, monitoring, network tool.

4 None Network

- Container không có networking.
- Không có địa chỉ IP, không routing.
- Cách ly hoàn toàn.

Container (None)
Không có IP!



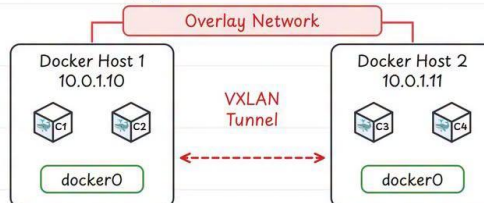
Không có network

Dùng khi:

Kiểm thử bảo mật, các workload nhạy cảm không được phép truy cập mạng.

5 Overlay Network

- Dùng cho giao tiếp giữa các container trên nhiều host.
- Tạo một network ảo trải rộng trên nhiều Docker host.
- Bên dưới sử dụng VXLAN.

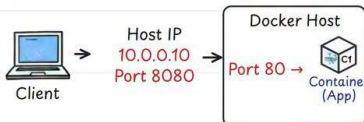


Dùng khi:

Swarm mode, Kubernetes, microservices chạy trên nhiều host.

6 Port Mapping (Publish Ports)

- Expose port của container ra host và thế giới bên ngoài.
- Định dạng: -p <host_port>:<container_port>



Ví dụ:

`docker run -d -p 8080:80 nginx`
Truy cập Nginx tại `http://10.0.0.10:8080`

7 Docker DNS (Embedded DNS)

- Docker cung cấp một DNS server nội bộ (127.0.0.11) để phân giải tên container.
- Container có thể tìm container khác bằng tên (trên user-defined network).



Ví dụ:

Container A gọi: `http://ga-api:8080`
DNS phân giải 'ga-api' thành 172.17.0.3

Điểm chính

- Bridge = network mặc định (cách ly nhưng vẫn kết nối)
- Host = hiệu năng tốt nhất
- None = không truy cập mạng
- Overlay = giao tiếp multi-host
- Port Mapping = expose service
- Docker DNS = service discovery bằng tên

Mẹo phòng vấn

Hiểu khi nào dùng từng network mode. Nhớ vững port mapping, phân giải tên container và overlay cho mô hình multi-host.

Lệnh thường dùng

```
docker network ls
docker network inspect <network>
docker run -d --network <network_name> nginx
docker run -d -p 8080:80 nginx
```

Ghi nhớ

- Container trên cùng network giao tiếp được bằng IP hoặc tên.
- Port mapping giúp expose port của container.
- Overlay giúp container trên các host khác nhau giao tiếp.

Kubernetes Networking

1 Pod Network

- Mỗi Pod có một địa chỉ IP riêng.
- Các Pod giao tiếp được với nhau giữa các node mà không cần NAT.
- Được triển khai bằng CNI plugin (Flannel, Calico, Cilium, Weave).

Điểm chính

Mọi Pod IP đều có thể truy cập từ mọi Pod khác trong cluster.

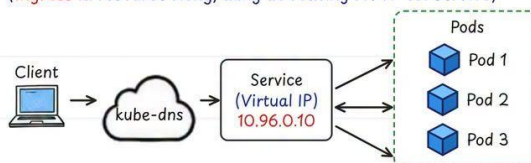
Pod Network (Flat Network)



Các Pod ở các node khác nhau giao tiếp trực tiếp.

2 Service (Tổng quan)

- Service cung cấp một endpoint ổn định (IP + DNS) cho một nhóm Pod.
- Tách client khỏi vòng đời (lifecycle) của Pod.
- Các loại Service: **ClusterIP** | **NodePort** | **LoadBalancer** (Ingress là resource riêng, dùng để routing HTTP tới Service)



3 ClusterIP (mặc định)

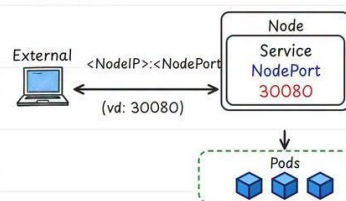
- Expose Service trên một IP nội bộ của cluster.
- Chỉ truy cập được bên trong cluster.
- Dùng cho giao tiếp nội bộ.



Không truy cập được từ bên ngoài cluster.

4 NodePort

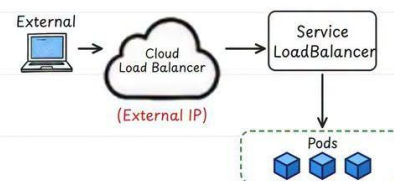
- Expose Service trên IP của mỗi Node tại một port tĩnh (30000-32767).
- Truy cập từ bên ngoài qua <NodeIP>: <NodePort>.



Traffic đi tới Node rồi mới tới Pod.

5 LoadBalancer

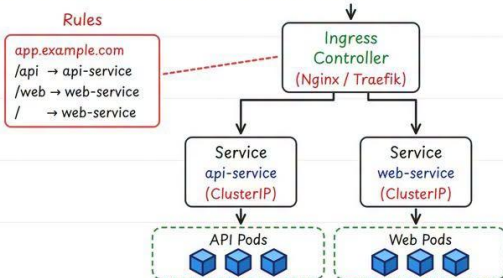
- Tạo một Load Balancer bên ngoài trên cloud (AWS ELB, Azure LB, GCP LB).
- Cung cấp một IP bên ngoài riêng.



Phù hợp nhất cho ứng dụng production.

6 Ingress

- Routing HTTP/HTTPS tới các Service dựa trên rule.
- Hoạt động ở Layer 7 (Application Layer).
- Cần có Ingress Controller (vd: Nginx Ingress).



Ghi chú DevOps

Dùng Network Policy để kiểm soát traffic Pod-to-Pod nhằm tăng bảo mật.

Ví dụ: chỉ cho phép API Pods truy cập DB Pods.

7 Tóm tắt các loại Service

Loại	Expose	Truy cập	Use Case
ClusterIP	Cluster Internal IP	Trong cluster	Service nội bộ
NodePort	Node IP + Port	Trong & ngoài	Dev / Testing
LoadBalancer	Cloud LB (External IP)	Bên ngoài	Ứng dụng production
Ingress	HTTP/HTTPS (L7)	Bên ngoài (qua rule)	Web app / API

Mẹo phòng vấn

- ClusterIP là mặc định, không truy cập được từ bên ngoài.
- NodePort expose trên mọi node - cần thận về bảo mật.
- LoadBalancer phụ thuộc vào cloud.
- Ingress = routing thông minh hơn (L7).
- Pod giao tiếp trực tiếp bằng Pod IP.

Ghi nhớ

- Service = endpoint ổn định
- Ingress = routing traffic thông minh
- Chọn đúng loại Service theo yêu cầu

Lệnh thường dùng

```
kubectl get pods -o wide
kubectl get svc
kubectl get ingress
kubectl describe svc <service-name>
kubectl describe ingress <ingress-name>
```


Cloud Networking (AWS VPC)

1 VPC là gì?

- Virtual Private Cloud (VPC) là một network được cách ly về mặt logic trên cloud.
- Bạn kiểm soát dải IP, subnet, route table, gateway và bảo mật.
- Cung cấp networking an toàn, dễ mở rộng và linh hoạt cho workload của bạn.

Điểm chính
VPC là network riêng của bạn trên AWS Cloud.

3 Public vs Private Subnet

Public Subnet

- ✓ Route ra Internet qua IGW.
- ✓ Resource có Public IP.
- ✓ Dùng cho bastion, ALB, web server.

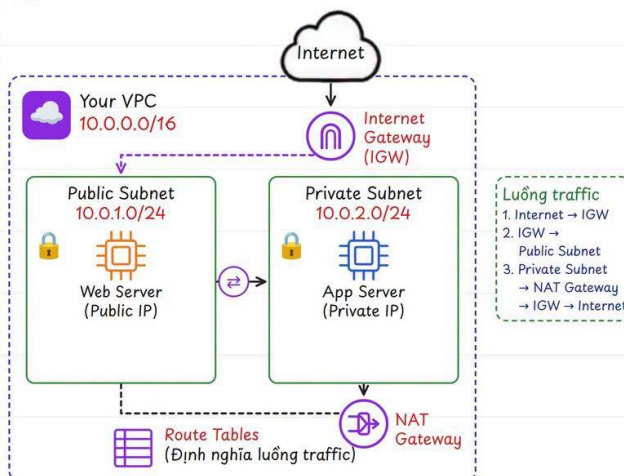


Private Subnet

- ✓ Không có route trực tiếp ra Internet.
- ✓ Resource chỉ có Private IP.
- ✓ Dùng cho app server, database, caching.



2 Tổng quan kiến trúc VPC



4 Internet Gateway (IGW)

- Mở rộng theo chiều ngang, có dự phòng và tính sẵn sàng cao.
- Cho phép giao tiếp giữa VPC và Internet.
- Mỗi IGW chỉ gắn với một VPC.

Ví dụ: Internet ↔ IGW ↔ Public Subnet

5 NAT Gateway

- Cho phép instance trong private subnet truy cập Internet.
- Chặn các kết nối inbound từ Internet.
- Dùng để cập nhật, tải xuống, truy cập package repository.

Ví dụ: Private Subnet → NAT → IGW → Internet

6 Route Tables

- Định nghĩa network traffic sẽ đi đâu.
- Được gắn với subnet.
- Áp dụng longest prefix match.
- Main Route (local): 10.0.0.0/16 → local

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxx (cho Public)
0.0.0.0/0	nat-xxxxxxx (cho Private)

7 Security Groups (Stateful Firewall)

- Hoạt động ở cấp instance.
- Stateful: return traffic được tự động cho phép.
- Chỉ có allow rule (mặc định deny tất cả).
- Dùng để kiểm soát chi tiết (fine-grained).



Ví dụ: Cho phép HTTP(80) và HTTPS(443) tới Web Server

8 Best Practices

- ✓ Đặt resource private trong Private Subnet.
- ✓ Dùng NAT Gateway cho outbound internet.
- ✓ Dùng Security Group để kiểm soát ở cấp instance.
- ✓ Dùng route table riêng cho các subnet khác nhau.
- ✓ Bật Flow Logs để giám sát traffic.
- ✓ Áp dụng quyền tối thiểu (least privilege).

★ Mẹo phòng vấn

- ★ IGW = Internet vào/ra VPC
- ★ NAT Gateway = chỉ outbound cho Private
- ★ Route Table quyết định đường đi
- ★ Security Group = Stateful, cấp instance
- ★ NACL = Stateless, cấp subnet
- ★ Public Subnet có route tới IGW
- ★ Private Subnet có route tới NAT Gateway

💡 Trường hợp sử dụng phổ biến

- ✓ Host web app public trong public subnet.
- ✓ Đặt application & database trong private subnet.
- ✓ Cho phép private instance tải update qua NAT.
- ✓ Tách biệt môi trường (Dev, QA, Prod) bằng các VPC riêng.

🔧 Lệnh hữu ích (AWS CLI)

- aws ec2 describe-vpcs
- aws ec2 describe-subnets
- aws ec2 describe-route-tables
- aws ec2 describe-security-groups
- aws ec2 describe-nat-gateways
- Liệt kê VPC
- Liệt kê Subnet
- Liệt kê Route Table
- Liệt kê Security Group
- Liệt kê NAT Gateway

💡 Ghi nhớ

- ✓ VPC = network riêng của bạn
- ✓ IGW = truy cập Internet
- ✓ NAT = Internet chiếu ra
- ✓ Route Table = đường đi của traffic
- ✓ SG = firewall stateful

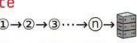
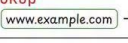
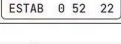
Cheat Sheet về Networking

1 So sánh chính

A. OSI vs TCP/IP			B. TCP vs UDP			C. HTTP vs HTTPS		
OSI Model (7 tầng)	TCP/IP Model (4 tầng)	Chức năng	Tiêu chí	TCP	UDP	Tiêu chí	HTTP	HTTPS
7. Application	Application	Dịch vụ mạng cho ứng dụng	Kết nối	Connection-oriented (3-way handshake)	Connectionless	Tên đầy đủ	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
6. Presentation			Độ tin cậy	Tin cậy, đảm bảo truyền	Best-effort, không đảm bảo	Port	80	443
5. Session			Thứ tự	Giữ đúng thứ tự packet	Không đảm bảo thứ tự	Bảo mật	Không mã hóa	Mã hóa (SSL/TLS)
4. Transport	Transport	Giao tiếp end-to-end	Tốc độ	Chậm hơn (overhead để đảm bảo tin cậy)	Nhanh hơn (overhead thấp)	Dữ liệu	Plain Text	Encrypted
3. Network	Internet	Đánh địa chỉ logic & routing	Kiểu dữ liệu	Stream of bytes	Datagram (packet)	Xác thực	Không	Có (Certificate)
2. Data Link	Network Access	Framing, đánh địa chỉ MAC	Use Cases	Web, Email, File transfer, SSH	DNS, VoIP, Gaming, Streaming	SEO / Độ tin cậy	Thấp hơn	Cao hơn (bảo mật)
1. Physical		Truyền bit qua môi trường vật lý	Port ví dụ	80, 443, 22, 3306	53, 123, 161, 500	Sử dụng	Web traffic thông thường	Dữ liệu nhạy cảm, API, đăng nhập

Mẹo: Dùng HTTPS ở mọi nơi. Nó mã hóa dữ liệu và tạo dựng niềm tin.

2 Lệnh thường dùng

Command	Mục đích	Ví dụ	Kết quả hiển thị
ping 	Kiểm tra khả năng kết nối (reachability) và độ trễ.	ping example.com [Windows: ping example.com]	Host có phản hồi không, thời gian phản hồi (latency) và tỉ lệ mất packet.
tracert 	Truy vết đường đi của packet tới đích.	tracert example.com [Windows: tracert example.com]	Tất cả các hop (router) và độ trễ tại mỗi hop.
nslookup 	Truy vấn DNS để lấy thông tin tên miền.	nslookup example.com	Địa chỉ IP, name server và DNS record.
dig 	Tra cứu DNS nâng cao (công cụ cho admin).	dig example.com dig @8.8.8.8 example.com A	DNS record chi tiết, TTL, response code, v.v.
curl 	Truyền dữ liệu tới/từ server (HTTP/HTTPS).	curl -I https://example.com/api curl -X GET https://example.com/api	HTTP header, response code và dữ liệu phản hồi.
ip 	Xem / cấu hình network interface và route.	ip a ip route	Địa chỉ IP, interface, routing table.
ss 	Hiển thị thông kê socket (kết nối TCP/UDP).	ss -tln ss -tp	Port đang lắng nghe, kết nối đang mở, thông tin process.

3 Ghi chú nhanh

Các port quan trọng (dùng nhiều nhất)			Ghi nhớ		Best Practices	
22	SSH (Secure Shell)	5432 PostgreSQL	✓	DNS chuyển domain thành IP.	✓	Áp dụng quyền truy cập tối thiểu.
53	DNS (Domain Name System)	27017 MongoDB	✓	IP định tuyến packet tới đích.	✓	Giữ security group chặt chẽ.
80	HTTP (Web)	6379 Redis	✓	TCP = tin cậy, UDP = nhanh.	✓	Giám sát network traffic & log.
443	HTTPS (Secure Web)	25 SMTP (Email)	✓	Firewall kiểm soát traffic nào được phép.	✓	Mã hóa dữ liệu khi truyền (HTTPS/TLS).
3306	MySQL	21 FTP	✓	Luôn dùng HTTPS để bảo mật.	✓	Ghi lại tài liệu kiến trúc mạng.